

Assignment 1 – Answers

Full Stack Development

Part A: Conceptual Questions

1.

Full Stack Development refers to the practice of designing and building all three layers of a web application: the **Frontend** (user interface), the **Backend** (server-side logic), and the **Database** (data storage). A **Full Stack Developer** is someone who can work across all these layers independently.

Responsibilities of a Full Stack Developer:

- Building and designing the user interface
- Writing server-side code and creating APIs
- Managing and querying databases
- Deploying and maintaining the web application

Example: A Full Stack Developer building Zomato would design the restaurant listing page (frontend), write the order-processing logic (backend), and manage food and user data (database).

2.

The **Frontend** is the part of a web application that users see and interact with directly in their browser. It includes everything visible — layout, buttons, images, forms, menus, and animations. The goal is to create a smooth and visually appealing user experience.

Technologies used in Frontend:

- **HTML** – Structures the content (headings, paragraphs, images)
- **CSS** – Styles the layout (colors, fonts, spacing)
- **JavaScript** – Adds interactivity and dynamic behavior
- **React / Angular / Vue.js** – Modern frameworks for building UIs

Example: On Netflix, the homepage showing movies, the search bar, and the play button — all of that is the frontend.

3.

The **Backend** is the server-side part of a web application that runs behind the scenes. Users cannot see it directly, but it powers all the actions — logging in, placing orders, loading personalized content, and more.

Main Responsibilities:

- Processing requests received from the frontend
- Applying business logic (e.g., calculating totals, checking stock)
- Communicating with the database to read or write data
- Handling user authentication and security
- Sending responses back to the frontend

Technologies: Node.js, Python (Django/Flask), Java, PHP, Express.js

Example: When you log into Instagram, the backend checks your credentials and sends back your profile data.

4.

A **Database** is an organized collection of data stored electronically, allowing information to be efficiently stored, retrieved, updated, and deleted. Without a database, a website cannot remember any information between sessions.

Why do websites need databases? Websites handle large amounts of dynamic data (user accounts, orders, posts) that must persist even after the browser is closed. Databases make this possible in a structured and efficient way.

Examples of data stored in a database:

- User names, email addresses, and passwords
- Product listings, prices, and stock levels
- Order history and payment records
- Blog posts, comments, and likes
- Video/movie details (e.g., YouTube, Netflix)

Types: MySQL, PostgreSQL (relational); MongoDB (non-relational)

5.

DNS (Domain Name System) acts like a phone book for the internet. It translates human-readable domain names (like `www.google.com`) into machine-readable IP addresses (like `142.250.195.4`) that computers use to locate servers.

How DNS helps users access websites:

1. User types `www.amazon.com` in the browser
2. Browser asks the DNS server for the IP address of `amazon.com`
3. DNS responds with the IP address (e.g., `54.239.28.85`)
4. Browser connects to that IP and loads the website

Analogy: DNS is like a contact list on your phone — instead of remembering a number, you search the name and it finds the number for you.

6.

An **IP (Internet Protocol) Address** is a unique numerical label assigned to every device connected to a network. It works like a home address — it tells the internet where data should be sent and from where it came.

Types:

- **IPv4:** 192.168.1.1 (older, 32-bit format)
- **IPv6:** 2001:0db8:85a3::8a2e:0370:7334 (newer, 128-bit format)

Why IP addresses are important in networking:

- They uniquely identify each device on the internet
- They allow data packets to be routed to the correct destination

- They enable communication between different devices and servers worldwide

Example: When you send a WhatsApp message, your device's IP address tells the server where to deliver the reply.

7.

Client-Server Architecture is a model where two parties communicate: the **Client** (requester) and the **Server** (provider). The client sends a request; the server processes it and sends back a response.

How they communicate:

1. The client (browser) sends an HTTP request to the server
2. The server receives and processes the request
3. The server queries the database if needed
4. The server sends an HTTP response (HTML, JSON, etc.) back
5. The client displays the result to the user

Example: When you open Gmail, your browser (client) requests your emails from Google's server, which fetches them from the database and returns them.

8.

An **API (Application Programming Interface)** is a set of rules that allows two software applications to communicate with each other. It defines how requests should be made and what format the responses will be in.

Role of APIs in web applications:

- They allow the frontend to communicate with the backend
- They enable integration of third-party services (Google Maps, payment gateways)
- They allow different apps to share data securely

Example: 'Login with Google' uses Google's API to verify your identity without sharing your password.

Analogy: An API is like a waiter in a restaurant — you give the order, the waiter takes it to the kitchen, and brings back your food.

9.

HTTP Methods are standardized actions used in web requests to tell the server what operation to perform. They form the basis of RESTful API communication.

- **GET** – Retrieves/reads data from the server without changing anything. *Example: Loading a product page fetches product details via GET.*
- **POST** – Sends new data to create a record on the server. *Example: Submitting a registration form sends data via POST to create a new account.*
- **PUT** – Updates an existing record on the server. *Example: Editing your profile sends a PUT request to update your data.*
- **DELETE** – Removes a specific record from the server. *Example: Removing an item from your cart sends a DELETE request.*

10.

Developer Tools (DevTools) are built-in browser utilities (opened by pressing F12) that let developers inspect, debug, and analyze web pages in real time.

Three important tabs:

- **Elements Tab:** Shows the HTML structure and CSS styles of the page. Developers can inspect any element, see its properties, and edit HTML/CSS live to test changes instantly.
- **Network Tab:** Displays all network requests made by the page (API calls, images, scripts). Developers can see request/response data, HTTP status codes, and loading times.
- **Console Tab:** Shows JavaScript logs, warnings, and errors. Developers use it to run JS code directly, debug errors, and monitor app behavior.

Part B: Diagram Activities

11.



Layer Descriptions:

- **User:** The person using the application through a browser or mobile app.
- **Frontend:** The visual interface users see and interact with. Built with HTML, CSS, JavaScript.
- **Backend:** Processes requests, applies business logic, and communicates with the database.

- **Database:** Stores all application data persistently — accounts, products, orders, etc.

12.

USER types URL in Browser

|

v

BROWSER sends HTTP request

|

v

DNS resolves domain name

(URL → IP Address)

|

v

SERVER receives the request

(Backend processes the logic)

|

v

DATABASE is queried for data

|

v

DATABASE returns data to Server

|

v

SERVER sends HTTP Response

(HTML, CSS, JS, JSON data)

|

v

BROWSER renders the page

|

v

WEBSITE DISPLAYED to User

Part C: Real-World Application Questions

13.

Frontend:

The student sees a registration form with fields like Name, Email, Password, and Course. When the student fills in the details and clicks 'Submit', the frontend validates the inputs (checks that email format is correct, no field is empty). If valid, it sends the data to the backend via an HTTP POST request.

Backend:

The backend server receives the form data. It further validates it (checks if email already exists), hashes (encrypts) the password for security, and stores the record in the database. It then sends

a response back — either 'Registration Successful!' or an error message.

Database:

The database permanently stores the student's record in a 'Students' table with fields: StudentID, Name, Email, HashedPassword, Course, and RegistrationDate. This data is retrieved later when the student logs in.

14.

Browser: The user types 'Laptop' in Amazon's search bar and presses Enter. The browser sends an HTTP GET request to Amazon's server with the search term as a parameter.

Server: Amazon's backend server receives the search request. It processes the query, applies business logic (filters by relevance, ratings, availability), and sends a structured query to the database to find all matching products.

Database: The database searches through millions of product records and returns all laptop listings — names, prices, images, ratings, and seller information — back to the server.

Response: The server formats the data and sends it as an HTTP response to the browser. The browser renders the search results page, displaying all matching laptops with images, prices, and ratings for the user to browse.

15.

Website Observed: www.google.com

- **Elements Tab:** The Google homepage is built with standard HTML. The search bar is an `<input>` element inside a `<form>` tag. The 'Google Search' button is a `<button>` element. All layout uses `<div>` containers with CSS classes for styling and positioning.
- **Network Tab:** Multiple network requests are made on page load — for the logo (PNG image), JavaScript files (.js), and CSS stylesheets. Each shows a 200 status code (success). Typing in the search box triggers additional XHR requests for autocomplete suggestions.
- **Console Tab:** No errors appear on Google's homepage, indicating clean JavaScript. Running `document.title` in the console returns 'Google', confirming JS execution works correctly.

Reflection Questions

16.

When a user types a URL like www.youtube.com and presses Enter, the browser first contacts a **DNS server** to convert that domain name into an IP address — the actual numerical address of the server hosting the site.

Using that IP address, the browser sends an **HTTP request** to the web server. The server receives the request, processes it (querying a database for dynamic content if needed), and sends back an **HTTP response** containing HTML, CSS, JavaScript, and data.

The browser then **renders** the HTML and CSS to paint the webpage on screen and executes JavaScript to make it interactive. This entire process — from typing the URL to seeing the full page — typically takes less than a second.

17.

Real-World Example: Zomato (Food Delivery App)

- **Frontend (What you see):** The Zomato app screen showing restaurant names, food photos, prices, the search bar, and the 'Order Now' button. Built with React Native / JavaScript, it is what the user directly interacts with. When you tap a restaurant, the frontend sends a request to the backend.
- **Backend (What happens behind the scenes):** When you place an order, the backend verifies your account, calculates the delivery charge, checks restaurant availability, processes payment through a payment API, and assigns a delivery partner. The user never sees this logic, but it powers everything.
- **Database (Where data is stored):** Stores all persistent data — user profiles, restaurant menus, food prices, order history, delivery partner locations, and reviews. When you check past orders, the backend fetches that data from the database and sends it to the frontend to display.

Summary: The Frontend is the face of the app, the Backend is the brain, and the Database is the memory.